



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:

Martes, 03/02/2026

Hora de inicio:

17:20

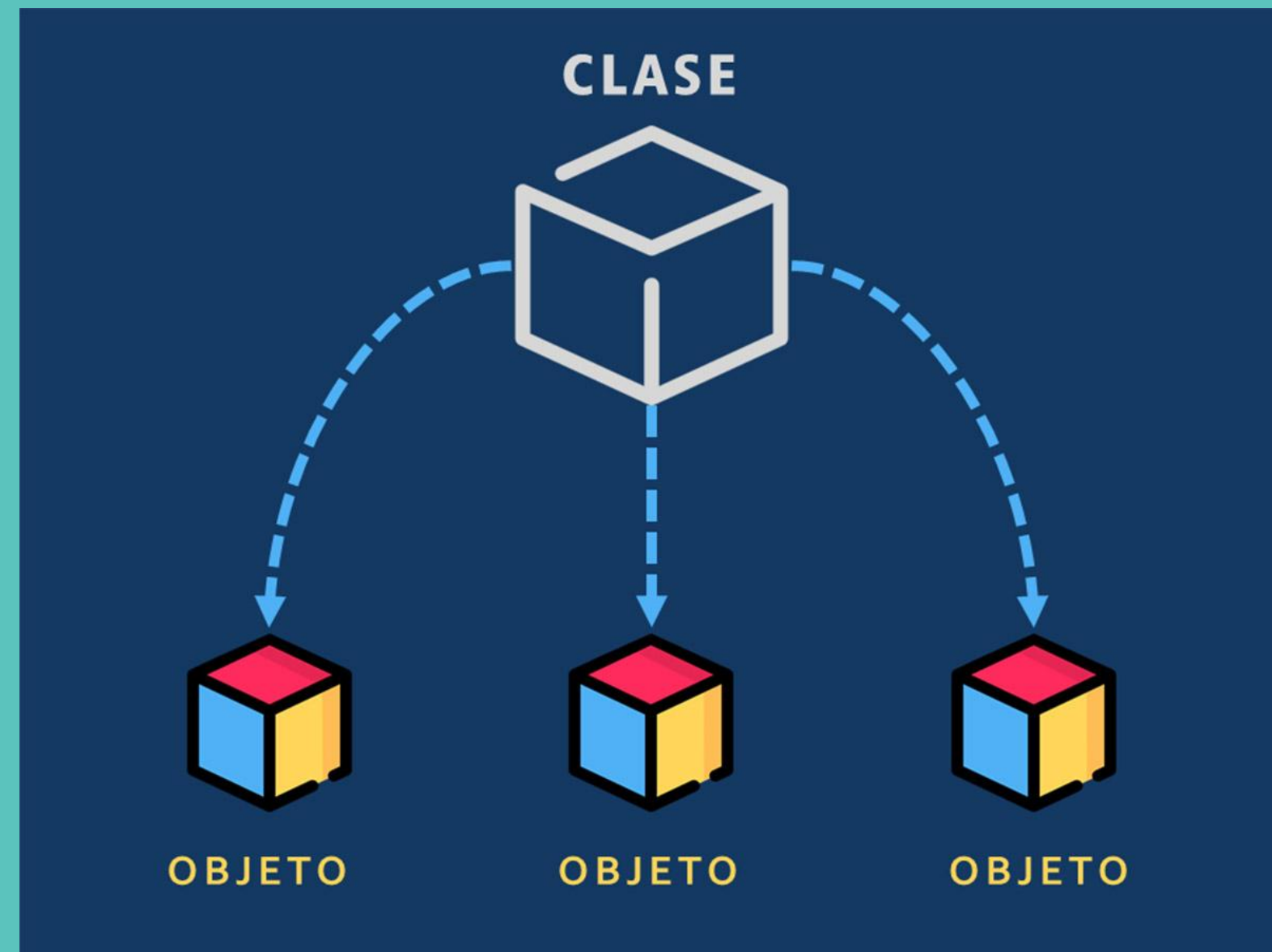
Introducción a la programación y computación 2 [B]

Diego Andres Huite Alvarez

AGENDA

- Revisar su asignación dtt
- Clase POO – fin unidad 1
- Ejemplos prácticos
- Dudas

PROGRAMACIÓN ORIENTADA A OBJETOS



PROGRAMACION ORIENTADA A OBJETOS

Es un paradigma de programación que utiliza "objetos" y sus interacciones para diseñar y programar aplicaciones. Los pilares de la POO son abstracción, encapsulamiento, herencia, polimorfismo.

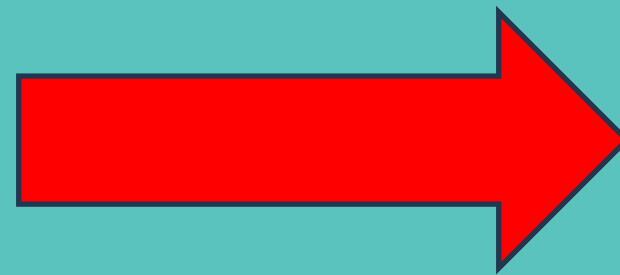
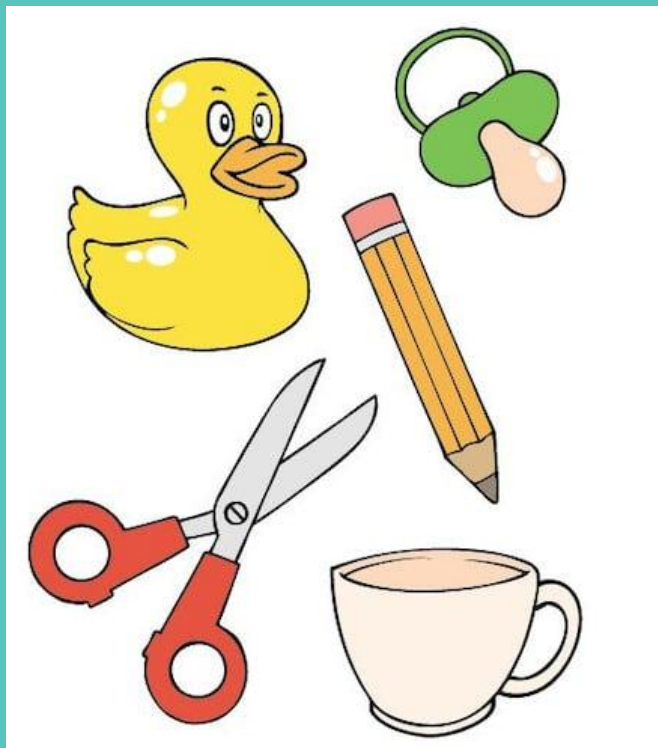
Elementos básicos de POO:

- Clases
- Objetos
- Atributos
- Métodos



PROGRAMACION ORIENTADA A OBJETOS

Los objetos son entidades que encapsulan datos y funcionalidades relacionadas, permitiendo modelar elementos del mundo real o conceptos abstractos dentro de un programa.



```
using System;

class MiClase
{
    public string Atributo;

    public MiClase(string parametro)
    {
        this.Atributo = parametro;
    }

    public void Metodo()
    {
        Console.WriteLine("¡Hola, soy un método de la clase!");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // Crear un objeto de la clase
        MiClase objeto = new MiClase("Hola");

        // Acceder a los atributos y métodos del objeto
        Console.WriteLine(objeto.Atributo);
        objeto.Metodo();
    }
}
```



Elementos básicos poo

- Clase: Una clase es una plantilla o un modelo a partir del cual se crean los objetos. Define un conjunto de atributos y métodos que los objetos de esa clase compartirán.

```
using System;

class Persona
{
    public string Nombre;
    public int Edad;

    public Persona(string nombre, int edad)
    {
        this.Nombre = nombre;
        this.Edad = edad;
    }

    public void Saludar()
    {
        Console.WriteLine($"Hola, mi nombre es {Nombre} y tengo {Edad} años.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Persona p = new Persona("Juan", 25);
        p.Saludar();
    }
}
```



Elementos basicos poo

- Objeto: Un objeto es una instancia de una clase. Es la entidad real que se crea en la memoria y se puede manipular.

```
Persona p = new Persona("Juan", 25);  
p.Saludar();
```



Elementos basicos poo

- Métodos: Los métodos son funciones dentro definidas de una clase que describen Comportamiento los objetos de esa clase.

```
public void Saludar()  
{  
    Console.WriteLine($"Hola, mi nombre es {Nombre} y tengo {Edad} años.");  
}  
  
public void cumplir_anos(){  
    this.Edad +=1;  
}
```



Elementos basicos poo

- Atributos: Los atributos son variables que pertenecen a una clase o a los objetos creados a partir de ella. Pueden ser atributos de instancia (específicos de cada objeto) o atributos de clase (compartidos por todos los objetos de la clase).



Elementos basicos poo

```
using System;

class Carro
{
    // Atributo de clase (estático)
    public static int Ruedas = 4;

    // Atributos de instancia
    public string Marca { get; }
    public string Modelo { get; }

    public Carro(string marca, string modelo)
    {
        Marca = marca;
        Modelo = modelo;
    }

    public void MostrarInfo()
    {
        Console.WriteLine($"Marca: {Marca}, Modelo: {Modelo}, Ruedas: {Carro.Ruedas}");
    }
}
```



Principios poo

- Encapsulamiento: El encapsulamiento es el principio de agrupar atributos y métodos que operan sobre esos datos dentro de una clase, restringiendo el acceso directo a algunos de los componentes.

```
class CuentaBancaria
{
    // Propiedad pública
    public string Titular { get; set; }

    // Campo privado (equivalente a __saldo en Python)
    private decimal _saldo;

    public CuentaBancaria(string titular, decimal saldoInicial)
    {
        Titular = titular;
        _saldo = saldoInicial;
    }
}
```



Principios poo

- Herencia: La herencia permite crear una clase nueva basada en una clase existente. Obteniendo atributos y métodos de la clase padre

```
using System;

// Clase base
class Animal
{
    public string Nombre { get; set; }

    public Animal(string nombre)
    {
        Nombre = nombre;
    }

    // Usamos 'virtual' para permitir que las clases hijas cambien el comportamiento
    public virtual void Hablar()
    {
        // Equivale al 'pass' de Python
    }
}

// Clase hija: Perro
class Perro : Animal
{
    public Perro(string nombre) : base(nombre) { }

    // Usamos 'override' para implementar nuestra propia versión
    public override void Hablar()
    {
        Console.WriteLine("Guau");
    }
}
```



Principios poo

- Polimorfismo: El polimorfismo permite que diferentes clases implementen el mismo método de diferentes maneras.

```
// Definimos la clase como 'abstract' porque no instanciaremos una "Forma" gen
abstract class Forma
{
    // Un método abstracto no tiene cuerpo; las hijas DEBEN implementarlo
    public abstract double Area();
}

class Rectangulo : Forma
{
    public double Ancho { get; set; }
    public double Alto { get; set; }

    public Rectangulo(double ancho, double alto)
    {
        Ancho = ancho;
        Alto = alto;
    }

    // Implementación específica para el rectángulo
    public override double Area()
    {
        return Ancho * Alto;
    }
}

class Circulo : Forma
{
    public double Radio { get; set; }

    public Circulo(double radio)
    {
        Radio = radio;
    }

    // Implementación específica para el círculo
    public override double Area()
    {
        return Math.PI * Math.Pow(Radio, 2);
    }
}
```



Principios poo

- Abstracción: La abstracción consiste en ocultar la complejidad y mostrar solo las funcionalidades esenciales objeto. del

```
using System;

// 'abstract' equivale a heredar de ABC en Python
abstract class FiguraGeometrica
{
    // @abstractmethod equivale a 'public abstract' en C#
    public abstract double Area();
}

// Heredamos de FiguraGeometrica
class Triangulo : FiguraGeometrica
{
    public double Base { get; set; }
    public double Altura { get; set; }

    public Triangulo(double @base, double altura)
    {
        Base = @base;
        Altura = altura;
    }

    // Implementamos el método obligatorio con 'override'
    public override double Area()
    {
        return (Base * Altura) / 2;
    }
}
```



Ejemplo poo



¿Dudas?

